



ML SYSTEM DESIGN

Counterfeit Goods Fraud Detection

An end-to-end ML system design for scoring ~60M e-commerce products for counterfeit risk — covering business goals, architecture, modeling, and the guardrail-driven threshold decision.

NORTH STAR

↓ Counterfeit goods

GUARDRAIL

Genuine upload rate flat

SCORING MODE

Batch, 1-2×/day

What we'll cover

From business problem to production-ready system



1. The fraud landscape

Mapping buyer, seller, attack, and logistics fraud



2. Problem framing

North Star metric and guardrails that bound the design



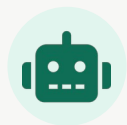
3. System architecture

Batch scoring, risk tiers, and the retraining loop



4. Entities & features

Product, seller, category, and context signals



5. Modeling & evaluation

The model ladder and metrics that matter for imbalance



6. Threshold selection

Tying the cutoff back to the business guardrail

The business problem



Sellers list counterfeit copies of genuine branded items — products made to imitate authentic goods and deceive customers into believing they're real.

Most susceptible categories

Electronics

Apparel

Beauty

Accessories

High demand, high value, and ease of replication drive risk in these categories.

METRICS THAT BOUND THE DESIGN

● North Star

Reduction in counterfeit goods on the platform

● Guardrail 1

Genuine-product upload rate stays constant

● Guardrail 2

Seller churn rate stays constant

Scale: ~60M products · Scoring: batch, 1-2x/day

Mapping the fraud surface

Four fraud categories on the platform — this design focuses on the seller-side branch

Buyer-side

Return fraud (swapped items) and refund fraud (claiming non-receipt while keeping the item)

Seller-side

THIS DESIGN

Counterfeit goods, bad-quality products, review manipulation, and fake listings

Attack-related

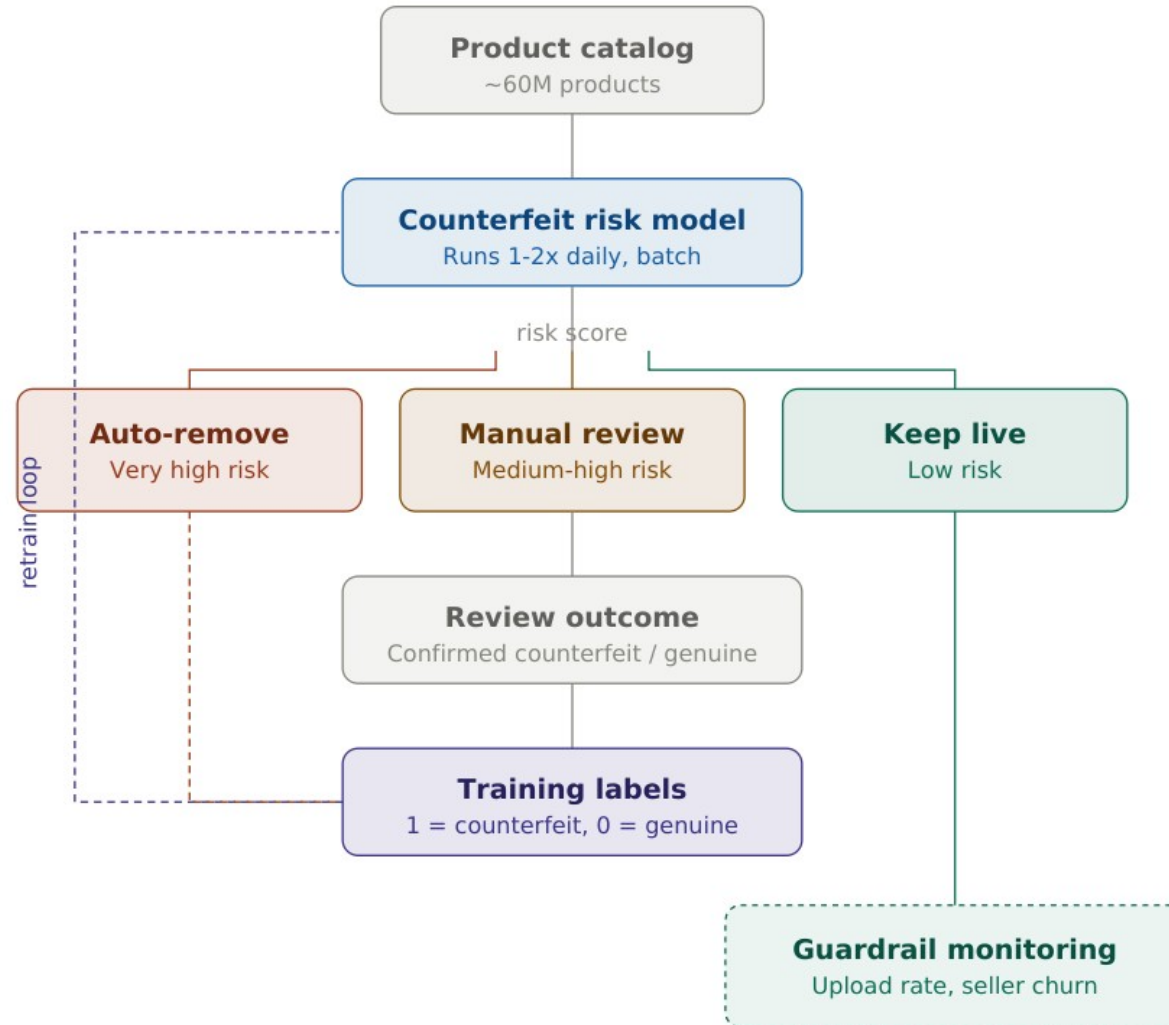
Account takeover (hacked buyer accounts) and automated bot attacks

Logistics-side

Delivery agent fraud and theft of items from packages in transit

System architecture

Every product is scored in batch. Risk tier determines what happens next — and outcomes always flow back to retrain the model.



Entities and features

Four entity groups feed the model — the feature engineering backbone



Product

Price, discount %, return/refund rate, counterfeit return ratio, image quality, image & text similarity to known counterfeits, brand-name similarity, celebrity-image flag, review sentiment signal



Seller

Reputation, rating, tenure on platform, number of listings, return rate, violation history



Category

Demand level, average value, counterfeit susceptibility, ease of replication, category return/incident rate



Context

Seasonal trend flag — counterfeit activity tends to rise during sales periods

The pipeline, step by step

Seven runnable stages — data through to model training (steps 1-4)

1

Data ingestion

`step1_simulate_data.py`

Pulls product, seller, category, and context entities — joining catalog DB, seller DB, returns logs, and an image-similarity service

2

Feature engineering

`step2_feature_engineering.py`

One-hot encodes category. Product/seller IDs excluded from tabular features — would feed an embedding layer in a neural net variant

3

Train / val / test split

`step3_train_test_split.py`

Stratified 70/15/15 split. Counterfeit rate is ~15% — stratifying preserves that ratio across all three sets

4

Model training

`step4_train_models.py`

Model ladder: logistic regression (interpretable baseline) → random forest → gradient boosting (strongest tabular performance)

The pipeline, step by step

Evaluation through to production tiering (steps 5-7)

5

Evaluation

`step5_evaluate_models.py`

Compares models on precision, recall, F1, and ROC-AUC — never accuracy, since the class is imbalanced (~85% genuine)

6

SENIOR-LEVEL DIFFERENTIATOR

Threshold selection

`step6_threshold_selection.py`

Scans the precision-recall curve for the lowest threshold that still satisfies a minimum precision target — maximizing recall without breaking the guardrail

7

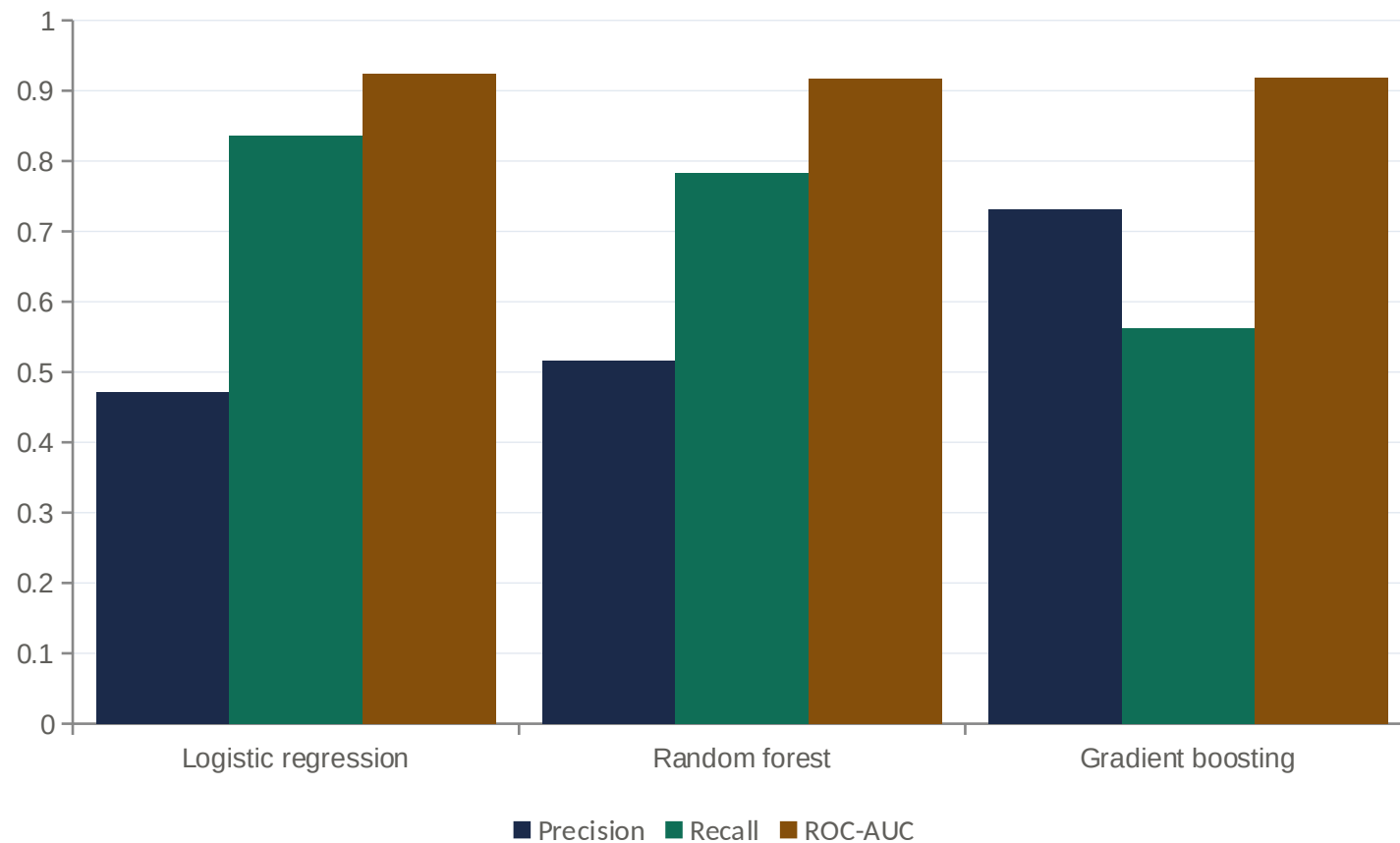
Risk tiering + final eval

`step7_risk_tiering_and_test_eval.py`

Maps scores to auto-remove / manual-review / keep tiers. Runs the one unbiased evaluation on the held-out test set

Model comparison

Validation set results — ROC-AUC selects the candidate, but precision protects the guardrail



WHY NOT JUST PICK THE BEST AUC?

Logistic regression wins on ROC-AUC and recall — but its precision (0.47) is far below the 0.85 guardrail needed to avoid wrongly banning genuine sellers.

The fix isn't a different model — it's a different threshold. See next: guardrail-driven threshold selection.



Guardrail-driven threshold selection

The decision that separates a junior answer from a senior one

JUNIOR APPROACH

“Pick the threshold that maximizes F1.”

Optimizes a generic statistical metric — ignores that auto-removal mistakes have an asymmetric, real business cost: banning a genuine seller.

SENIOR APPROACH

“Find the threshold with max recall, subject to precision ≥ 0.85 .”

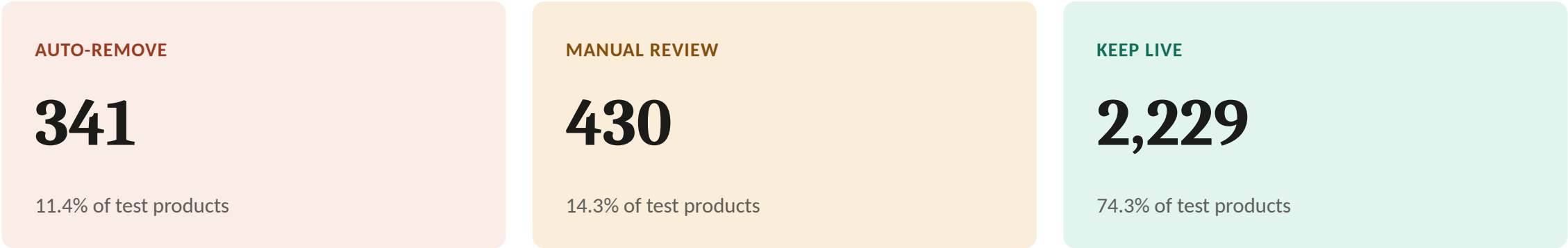
Ties the operating point directly to the guardrail metric defined at the start of the design — maximizing counterfeit catch rate without violating the constraint on genuine sellers.

RESULT ON VALIDATION SET

Threshold = 0.919 → precision 0.850, recall 0.451

Risk tiering on the test set

The one-time, unbiased evaluation — held-out test set, touched only here



Confusion matrix (at operating threshold)

	Pred: genuine	Pred: counterfeit
Actual: genuine	2,530	30
Actual: counterfeit	250	190



What stays verbal, not coded

Things to discuss out loud — interviewers want to hear the reasoning, not just see it implemented



Online A/B experimentation

Control = existing methodology, treatment = new model. Null/alternative hypothesis, significance level, p-value, Type I/II errors — before scaling to 100% of traffic.



Production monitoring

Feature drift (e.g. PSI) and label drift, scheduled retraining, and alerting if precision on audited “keep” samples degrades over time.



ID embeddings

In a neural net variant, seller_id and product_id would feed an embedding layer instead of being dropped — learning latent seller/product risk.



Feedback loop

Manual review outcomes and auto-removed high-risk items both flow back into the next training run, closing the loop shown in the architecture.



Key takeaways

- 1 Frame fraud problems by North Star + guardrail metrics before touching a model.
- 2 Scale and latency requirements decide batch vs. real-time architecture — don't default to real-time.
- 3 Use the right metrics for imbalance: precision, recall, F1, ROC-AUC, never plain accuracy.
- 4 The threshold is a business decision, not a statistics exercise — tie it to the guardrail.
- 5 Model output should map to a tiered action (auto / review / keep), not a binary call.